```
  def activated(total: Int): Behavior[Command] =
    Behaviors.receive { (context, message) =>
      message match {
        case Increase(amount) =>                    ❶
          val current = total + amount
          context.log.info(s"increasing to $current")
          activated(current)
        case Deactivate =>                          ❷
          deactivated(total)
        case Activate =>                            ❸
          Behaviors.same                            ❸
      }
    }
}
```

❶ Increases the total and logs

❷ Changes the behavior to deactivated

❸ Keeps the same behavior with the same total

You may wonder why on `Activate`, the wallet uses `Behavior.same` and not `activated(total)`. You can use `Behavior.same` because there is no incrementing, so the wallet is using the same behavior again. You don't need to pass a new `total`. (You could pass the same `total`, but it's easier to read `Behavior.same`.)

The `deactivated` behavior works as follows. It logs that it cannot increase when it receives an `Increase` message and returns to `activated` when it receives an `Activate`. When the wallet is `deactivated`, the `total` stays the same.

Listing 2.7 `deactivated` behavior

```
object WalletOnOff {

  ...

  def deactivated(total: Int): Behavior[Command] = {
```